

Projet Deep Learning : Débruitage de la parole

Simon RIOU | Hamza BOUASRIA
3A SICOM IMMAC

2026 — Mots-clés : deep learning, débruitage, parole, signal

Résumé — Ce rapport rend compte des travaux menés par notre groupe pour la réalisation du modèle DenoiseNet, dans le cadre d'un projet de deep learning dédié au débruitage de la parole. Il présente successivement la méthodologie employée pour la constitution de la base de données, les choix relatifs aux architectures de réseaux et aux hyperparamètres, ainsi que les résultats obtenus. Ces performances sont enfin analysées au regard des coûts énergétiques associés aux différentes expérimentations.

1 Construction d'une base de données

Les méthodes employées par la suite étant de nature supervisée, la constitution préalable d'une base de données d'entraînement s'est avérée nécessaire. À cette fin, nous avons choisi d'utiliser le corpus LibriSpeech ASR¹, et plus précisément le sous-ensemble de 2,25 heures de parole en anglais, exempt de bruit (`test-clean.tar.gz` [346M]). Concernant le bruit, nous nous sommes focalisés sur le cas d'une voix immergée dans un environnement de conversations multiples (bruit dit « babble »). Nous avons ainsi sélectionné plusieurs extraits issus de la base de données MS-SNSD (Microsoft Scalable Noisy Speech Dataset)², correspondant à des bruits de conversation (fichiers `noise_test/Babble_[1-8].wav`).

Après rééchantillonnage des signaux de parole et de bruit à une fréquence de 16 kHz, nous avons choisi de travailler avec des enregistrements bruités présentant un rapport signal sur bruit (SNR) de 0 dB.

Un point important a ensuite concerné la définition du processus de génération des signaux bruités à partir des signaux de parole et de bruit. Les principales contraintes étaient liées aux caractéristiques de la machine utilisée pour l'entraînement des modèles, notamment une capacité de stockage limitée et l'absence de bibliothèques Python permettant un chargement aisé de fichiers audio. Pour ces raisons, **nous avons opté pour le stockage séparé des signaux de parole et de bruit**, compressés au format `int16` et enregistrés sous forme de tenseurs `.pt`, directement exploitables par `PyTorch`.

Ce choix, motivé initialement par des considérations de stockage, a conduit a posteriori à une modification du protocole d'entraînement ayant un impact positif significatif sur les performances des modèles. En effet, **à chaque époque d'apprentissage, un même signal de parole est combiné avec un extrait de bruit différent**. Cette stratégie

1. <https://www.openslr.org/12/>

2. <https://github.com/microsoft/MS-SNSD>

accroît notablement la capacité de généralisation du modèle, puisqu'un même signal de parole est exposé à une diversité de conditions de bruit au cours d'un unique entraînement.

Les détails concernant l'implémentation du processus de bruitage des signaux peuvent être trouvés dans les fichiers `src/training/dataset` et `src/utils/constants` du dépôt GitHub.

2 Architecture CNN & masques

2.1 Généralités

Le modèle retenu pour cette partie est basé sur l'architecture U-Net. Cette dernière est particulièrement efficace pour la prédiction de masques. En effet, les signaux de parole présentent des motifs à différentes échelles :

- à l'échelle locale, on retrouve les trajectoires des formants et les harmoniques ;
- à moyen terme, on observe les changements d'énergie dus aux phonèmes et syllabes ;
- à grande échelle, on a la continuité de la voix et la présence de la parole au cours du temps.

Les différents blocs encodeurs du U-Net permettent de capturer les motifs à différentes échelles et donc d'apprendre des poids plus robustes qui généraliseront mieux à différents signaux de parole.

De plus, les « skip connections » relient directement des caractéristiques fines de l'encodeur vers le décodeur, et permettent ainsi de combiner des détails locaux avec de l'information globale sur le signal de parole. Ceci permet d'éviter des masques « flous » ou des erreurs isolées qui seraient très audibles dans le signal débruité.

Enfin, un autre avantage de la structure U-Net / CNN est la généralisation à différentes tailles de signaux. En effet, les paramètres convolutifs définissent la taille du champ récepteur (kernel) ainsi que la distance qu'il parcourt entre chaque convolution (stride) et la gestion des bords (padding), mais la taille de l'image, elle, peut varier d'un signal à l'autre sans changement nécessaire de modèle.

2.2 Le modèle DenoiseNet

Le modèle que nous avons développé comporte quelques particularités qui permettent de hautes performances en débruitage. Il est notamment important de noter quelques détails concernant la structure des blocs :

- on a deux convolutions par bloc, chacune d'entre elles étant « factorisée » (on effectue la convolution sur deux kernels rectangulaires plutôt que sur un kernel carré, cela aide à capturer différents détails selon les axes horizontal et vertical) ;
- on utilise la normalisation de groupe (GroupNorm) plutôt que celle de batch (BatchNorm). La normalisation de groupe calcule des statistiques au sein de chaque échantillon, plutôt que sur un batch entier. Elle est donc indépendante de la taille des batches et déterministe à l'entraînement et à l'inférence. Pour de petites

tailles de batch (typiquement ici 16 ou 32), cela améliore les performances sur des conditions de bruit inconnues (ce qui est crucial étant donné qu'on change le bruit associé à un signal de parole d'une époque sur l'autre!). Enfin, la normalisation de groupe a tendance à mieux converger sur une architecture comportant des skip connections.

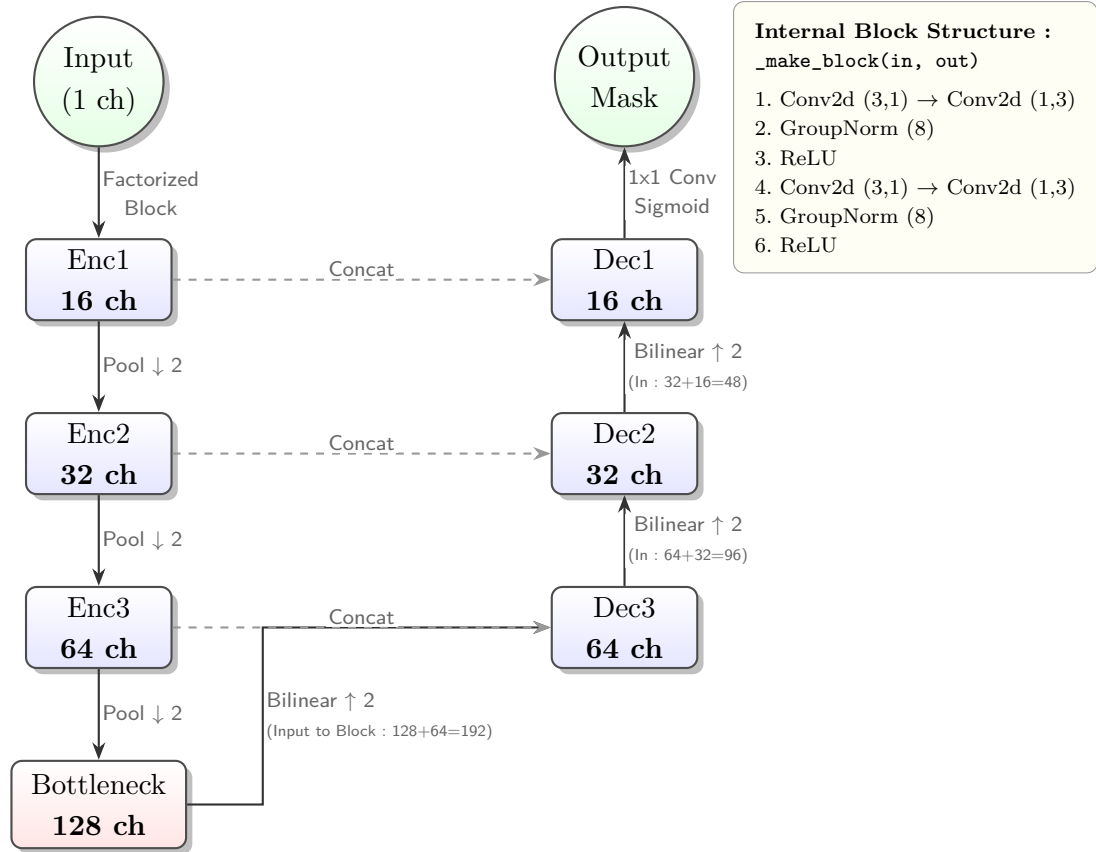


Figure 1 : Architecture utilisée (modèle DenoiseNet).

3 Entraînement - DenoiseNet

Cette section détaille les paramètres utilisés lors de l'entraînement de notre modèle.

3.1 Hyperparamètres

Comme expliqué précédemment, le principal atout de notre modèle est qu'il voit, au sein d'un même entraînement, différents échantillons de bruit sur un même signal de parole. Cela est rendu possible par l'utilisation d'une classe personnalisée pour charger les éléments du dataset lors de l'entraînement (voir fichier `src/training/dataset`). Cette classe nous permet de :

- envoyer différents échantillons de bruit à chaque époque ;

- communiquer différentes caractéristiques des échantillons du dataset (spectrogramme d’amplitude bruité ou propre, phase, masque idéal, etc.) ;
- économiser l’espace de stockage.

De nombreux changements d’hyperparamètres ont été réalisés durant le projet. Ci-dessous se trouve une table récapitulative des derniers utilisés.

Parameter	Value
SR	16 kHz
SNR	0 dB
NFFT / WINDOW	512
HOP_SIZE	256
MIN_DB_CLIP	-80 dB
LOSS	$\text{BCE} + 0.8 \text{ L1}_{\text{lin}} + 0.5 \text{ L1}_{\text{mel}} + 0.8 \text{ L1}_{\text{waveform}}$
EPOCHS	100
BATCH	32
LR	0.001

Table 1 : Configuration des hyperparamètres d’entraînement

Nous avons décidé de travailler avec des échantillons à 0 dB de SNR, car il nous paraissait inutile de débruiter des échantillons dont la qualité était supérieure à 0 dB à l’aide de méthodes de DL, ces derniers pouvant être traités efficacement par des méthodes habituelles.

Le paramètre MIN_DB_CLIP correspond au niveau d’énergie des spectrogrammes de puissance en deçà duquel on considère le silence. Cela permet d’éviter des valeurs limites aberrantes lorsqu’on calcule le logarithme du spectrogramme, et de normaliser ses valeurs entre 0 et 1 de manière consistante.

Les paramètres de STFT, notamment la taille de fenêtre (32 ms), permettent de capturer la parole sur des périodes où elle est quasi-stationnaire, tout en gardant les motifs des phonèmes et des syllabes. De plus, avec des signaux d’environ 3 secondes et des batchs de taille 32, les tenseurs sont environ de taille (32, 1, 257, 188), ce qui correspond à des dimensions raisonnables pour un modèle de type U-Net.

Nous avons décidé d’utiliser plusieurs fonctions de perte différentes lors de l’entraînement :

- BCE (Binary Cross-Entropy) : entre le masque prédit et le masque idéal. Elle permet de définir les régions qui contiennent de la parole et celles qui contiennent du bruit, mais ignore les erreurs *dans* les bins de parole et peut introduire des biais dans les amplitudes prédites.
- Perte L1 linéaire L1_{lin} : entre le spectrogramme de puissance débruité et celui du signal de parole pur. Elle permet de stabiliser l’entraînement (perte L1 non quadratique) et elle pénalise mieux les erreurs d’amplitude dans le spectrogramme.
- Perte L1 de mel L1_{mel} : sur 80 bandes de filtres de mel, priorise la perception humaine. Elle réduit les artefacts audio et améliore la qualité perçue.
- Perte L1 sur la forme d’onde $\text{L1}_{\text{waveform}}$: ajoute une cohérence au débruitage, va

au-delà du traitement bin par bin. Même en reconstruisant avec la phase du signal bruité, on gagne aussi en qualité perçue. Elle vient cependant au coût de devoir effectuer une STFT inverse à chaque batch entraîné.

Les coefficients devant chacune des pertes ont été optimisés en fonction de l'ordre de grandeur de chacune.

Enfin, les paramètres plus généraux de l'entraînement (nombre d'époques, taille du batch, learning rate) ont été définis empiriquement, au fur et à mesure des entraînements.

3.2 Entraînement

Cette section est dédiée à l'étude d'un entraînement réalisé avec la dernière version de notre modèle, sous la configuration d'hyperparamètres présentée plus haut.

Nous ne traçons plus bas que l'évolution des pertes de validation correspondant à chaque type de perte séparément. Il est important de noter que les fonctions de perte d'entraînement sont dynamiques (normalisées à chaque époque par rapport à la moyenne de chaque type de perte). Par conséquent, la perte d'entraînement n'est pas monotone et ne doit pas être interprétée comme une mesure de convergence. Cependant, les valeurs des pertes de validation n'étant pas normalisées, elles sont indicatrices de convergence.

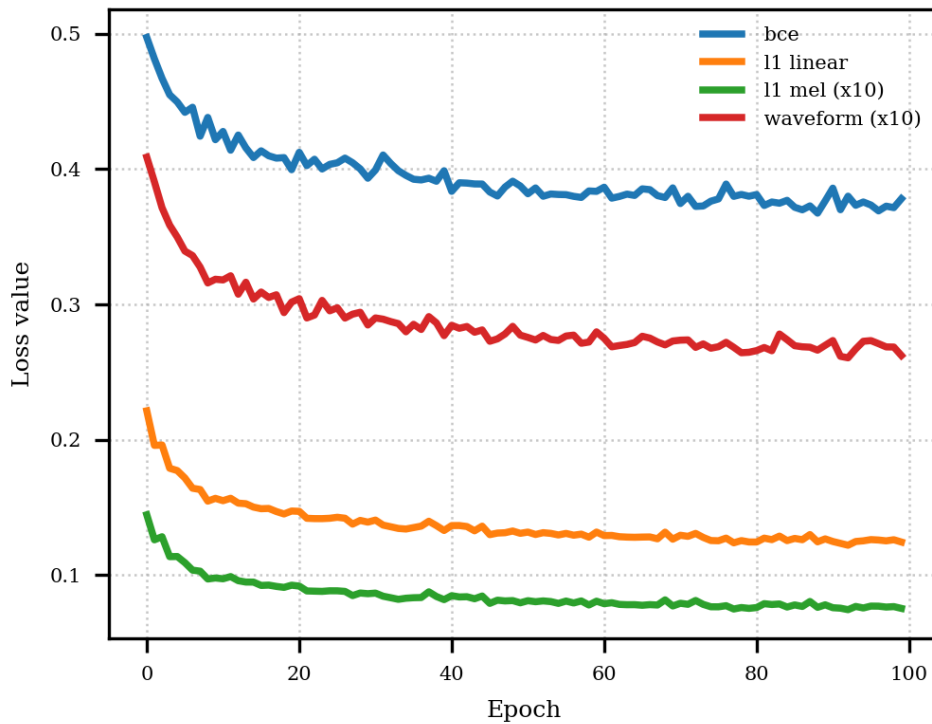


Figure 1 Validation loss curves. Evolution of the individual validation loss components over the course of training.

La multiplication par 10 des pertes Mel et forme d'onde est purement destinée à améliorer la visibilité du graphique.

Les quatre types de validation présentent une claire tendance à la baisse, ce qui est un bon signe de convergence du modèle. On observe notamment que les variations dans

la perte BCE se retrouvent (quasiment) à l'identique dans la perte de forme d'onde. En effet, les améliorations de la BCE entraînent une amélioration globale de la distribution d'énergie dans le spectre, et donc aussi dans la forme d'onde.

De même, le fait que les pertes L1 linéaire et de mel soient quasi parallèles suggère que le modèle ne surapprend pas sur une représentation au détriment d'une autre.

Pour terminer, nous avons conduit les entraînements sur la plateforme de calcul de l'ENSIMAG, plus précisément sur des GPUs (A40 ou A100). Un entraînement complet (100 époques) dure entre 30 et 35 minutes à 300 W de puissance, ce qui équivaut à environ 0,15 / 0,18 kWh. Sur la grille énergétique française, cela équivaut à environ 30 g CO₂eq / kWh (en prenant l'estimation moyenne), donc entre 4,5 et 5 g CO₂ par entraînement de 100 époques. À titre de comparaison, un trajet de 2 km en voiture thermique consomme entre 400 et 600 g de CO₂.

4 Résultats

Rendre compte qualitativement de l'amélioration de l'intelligibilité d'un signal de parole n'est pas facile. En effet, la valeur du ratio signal sur bruit est un premier indicateur, mais la perception et l'écoute restent les métriques les plus efficaces pour vérifier la qualité du débruitage.

C'est pour cela que nous avons sélectionné des échantillons de parole de test, issus d'une base de données différente (de sorte que le modèle ne les ait jamais vus, même en validation), sur lesquels réaliser l'inférence. Un aperçu des résultats est disponible sur cette archive : <https://simonriou.github.io/DenoiseNet/>.

L'amélioration du SNR est évidente. On gagne en moyenne (sur une centaine d'échantillons tests issus des datasets LJ Speech³ et Russian ASR⁴) 6 dB.

De plus, nous avons trouvé intéressant de comparer le débruitage réalisé avec une version intermédiaire de notre modèle contre la dernière version. La version intermédiaire n'ajoutait pas un échantillon de bruit différent à chaque époque d'un signal de parole, et n'utilisait qu'une fonction de perte BCE simple. On voit très clairement la meilleure généralisation du modèle lorsqu'il voit plusieurs bruits différents à chaque époque, ainsi que l'amélioration du débruitage et de la qualité audio perçue avec l'ajout des fonctions de perte de perception et de forme d'onde.

Enfin, les spectrogrammes des différents signaux audio (parole pure, parole bruitée, parole débruitée) permettent de visualiser les filtres prédits.

3. <https://keithito.com/LJ-Speech-Dataset/>

4. https://github.com/snakers4/open_stt

5 Conclusion et perspectives

Ce projet nous a permis de réaliser un modèle très compétent, dont l'inférence est utilisable en temps réel pour le débruitage de la parole noyée dans la conversation. La faible taille des données d'entraînement (environ 2,5 h de parole) a été compensée par l'ajout de différents échantillons de bruits à chaque époque de l'entraînement du modèle, et par le choix de fonctions de perte judicieuses, adaptées à la reconstruction audio et au traitement de la parole.

Les principales faiblesses du modèle résident en son incapacité à généraliser à d'autres types de bruits (il faudrait varier les échantillons de bruits lors de l'entraînement et probablement augmenter légèrement la complexité du modèle), ainsi que la faible qualité de la reconstruction audio due à l'utilisation de la phase bruitée. Ce dernier point n'a pas été grandement amélioré avec l'utilisation d'algorithmes de reconstruction classiques (du type Griffin & Lim). Par conséquent, une possible solution serait l'utilisation de modèles pré-entraînés de vocodeurs neuronaux lors de la reconstruction des signaux.

Enfin, ce modèle ouvre la voie vers de possibles applications dans des systèmes temps réel, tels que l'intégration à des systèmes d'assistance audio pour les personnes malentendantes, des logiciels de visioconférence ou de reconnaissance de la parole.